[DISTRIBUTIONAL]

Model Evaluation
From ML to GenAI

Erin LeDell, Ph.D.
Chief Scientist, Distributional
June 13, 2025

[DISTRIBUTIONAL]

01 Introduction
02 Traditional ML vs. GenAI
03 Eval & Benchmarking in ML
04 Modern AI Applications
05 GenAI Eval
06 LLM Eval in R

01 Introduction 👋

A little HeRstory:

- Working in Machine Learning since 2008, LLMs/GenAI since 2022
- Long time user of R ML packages: caret, tidymodels, mlr, etc.
- Author/co-author of a few packages for ML in R:
  - SuperLearner
  - subsemble
  - cvAUC
  - h2o (H2O AutoML)
  - h2o4gpu
- And one health related package (RIP 🪦): rHealthDataGov in rOpenHealth
- Research in the medical domain during my PhD research (Biostats)
- Worked with many medical/health companies during my time at H2O.ai
- Long history of benchmarking/eval in ML (e.g. AMLB)
- More info on https://github.com/ledell

02 Traditional ML vs. GenAI

Traditional ML vs. Generative AI

Traditional Machine Learning (ML):

- **Traditional ML** focuses on prediction of a **numeric** (regression) or **categorical** (classification) outcomes.
- It also includes **unsupervised techniques** such as **clustering**, **dimensionality reduction**, and **anomaly detection**.
- Mostly commonly applied to **tabular data**, however can also be applied to **text/image tasks** with appropriate preprocessing or feature extraction.
- Popular algorithms include: **Random Forest**, **GBM**, **GLM**, **SVMs**, and basic deep learning algorithms like multilayer perceptrons (**MLP**)

Example health/medical applications:

- Heart disease prediction on vital signs and medical history.
- Diabetes risk prediction based on BMI, age, glucose levels, etc.
- Cancer diagnosis (e.g., breast cancer) from diagnostic measurements or images.

Traditional ML vs. Generative AI

Generative AI:

- **Generative AI is designed to create new data** that resembles the training data. It doesn't just classify or predict — it can generate text, images, music, code, etc.
- When apply instruction tuning to a base next token prediction model like GPT-4, you get an **instruction following model** – such as **ChatGPT** – that is optimized for dialogue and task completion.

Example health/medical applications:

- Clinical note-taking tools that summarize and structure visit notes automatically.
- Drug discovery and molecular design (e.g. Insilico, AlphaFold3).
- RAG (LLMs + academic papers) for supporting Research Scientists.
- Generate synthetic, privacy-preserving images of rare pathologies for training.

Properties of traditional ML systems

Machine Learning system (with no GenAI) properties:

- ML models are usually **deterministic**:
  - Same input yields same output (with some exceptions like multi-threaded SGD).
- 📊 You need the **input/feature distribution to remain consistent** over time:
  - Otherwise you should re-train 🔄 the model.
- Prediction distribution should also be monitored (numeric or categorical).

03 Eval & Benchmarking in ML

Eval & Benchmarking in ML

Machine Learning has a long history of model evaluation and benchmarking. The off-the-shelf **ML model evaluation metrics are well understood**.

- **Supervised ML:** Classification and Regression
- **Unsupervised ML:** Clustering, Dim Reduction, etc.

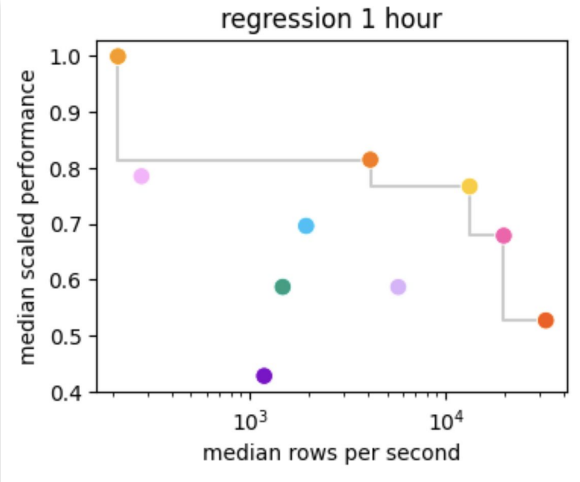
Eval vs Benchmarking:

- In ML, **"Eval" traditionally means model evaluation.**
- Eval / model evaluation occurs on one or more datasets.
- **Benchmarking is Eval across a large number of diverse datasets** to get a generalized estimate of the performance capabilities of an algorithm.
- There are a number of different evaluation metrics which are useful in different contexts (AUC, RMSE, etc).
- **ML Eval metrics are cheap**, though **benchmarking can be expensive**.
- Benchmarking is offline where as evaluation can be offline or online.

Benchmarking in ML: AutoML Benchmark

An example of a ML benchmarking framework is the AutoML Benchmark (AMLB):

- Compare the 10-fold CV performance of different Algorithms across different tasks: binary & multiclass Classification, regression (AUC, Log Loss, RMSE)
- List of model metrics that are tracked:
  - AUC
  - Log Loss
  - RMSE
  - Training time
  - Inference speed
- Pareto frontiers (e.g. AUC vs. Inference speed)



Offline vs Online Eval

- **Offline** Evaluation:
  - Eval that occurs in **development** or **deployment** environments.
  - In service of model selection / model upgrading for an ML application.
  - Includes benchmarking.

- **Online** Evaluation:
  - Eval that occurs in the **production** environments.
  - In service of model monitoring, drift detection.

Both can include:

- Evaluation with off-the-shelf model perf based metrics & also business metrics.
- Evaluate model fairness and bias.
- Evaluation of latency / inference speed.
- Evaluation of model memory / compute requirements.

Offline Eval in ML

Evaluation in the **development process** of ML systems is called Offline Eval.
The metrics that are commonly used in ML Eval are:

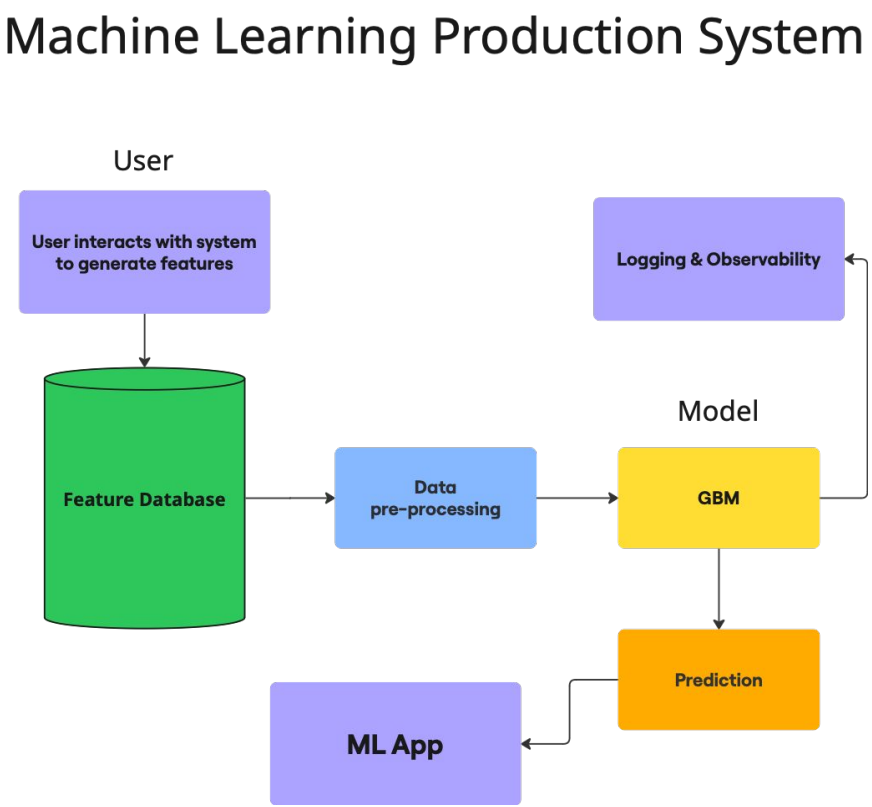
- Classification Tasks:
  - Accuracy (if classes are balanced)
  - Precision, Recall, F1-score
  - AUC-ROC / Precision-Recall AUC
  - Log Loss
- Regression Tasks:
  - Mean Squared Error (MSE) / Root Mean Squared Error (RMSE)
  - Mean Absolute Error (MAE)
  - R-squared
- Training time & energy resources
- Inference speed (More complex/accurate models can be slower at inference)

Online Eval in ML

Evaluation **in production** ML systems:

- Instrumented within MLOps systems
- Evaluate performance of production ML systems over time
  - **Evaluate predictions over time**
  - **Evaluate input data drift**: Komolgorov-Smirnoff (KS stat), Chi-squared Test
  - Trigger alerts when metrics fall below thresholds.
- Includes A/B testing and Shadow testing
  - A/B testing: Expose different models to subsets of users and compare results
  - Shadow testing: ML model deployed alongside live model w/o affecting user
- Feedback loops: Under some scenarios, model predictions influence the data it will later see.

ML System Diagram



04 Modern AI Applications

Modern AI (GenAI) Applications

Common design patterns for GenAI applications:

- **Q&A Chatbot** (ChatGPT, Claude, Gemini, etc)
- **Retrieval Augmented Generation (RAG)**
  - Augment an LLM with a knowledge base (document corpus or db) - combines a retriever with a generator.
  - Uses information retrieval techniques such as document embeddings to surface relevant context to ground the model's response in external knowledge.
- **Structured output**
  - Tool or function calling (agents)
  - Data extraction from text
  - Interfacing with APIs

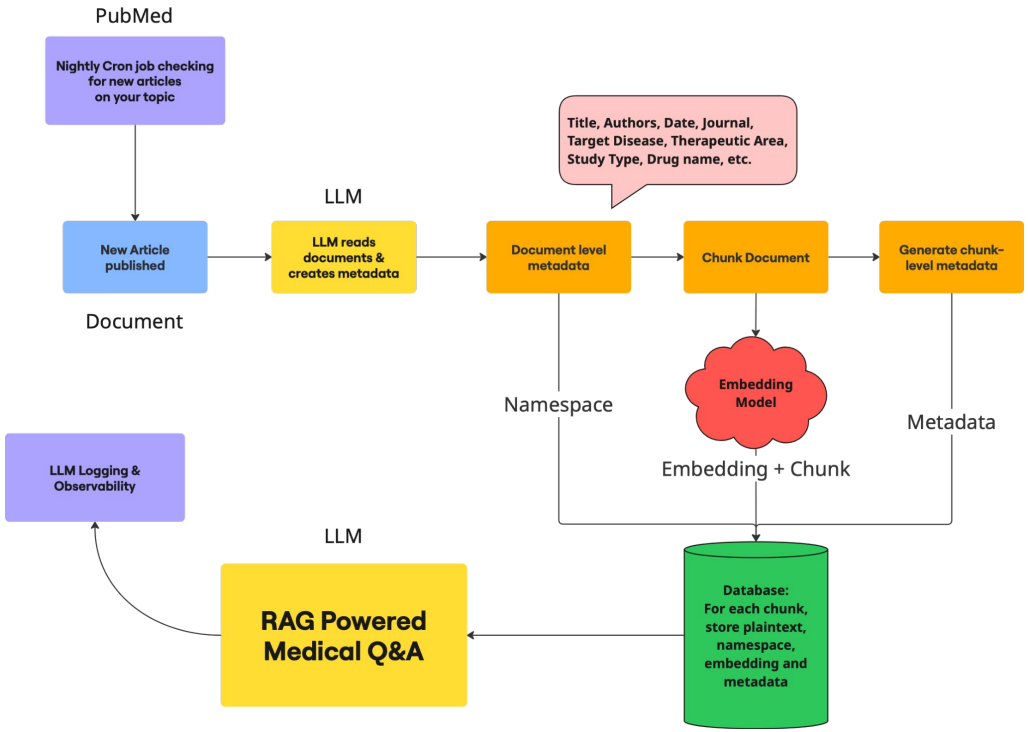
Modern AI Applications

- Modern AI Applications are **complex** and increasingly **use GenAI.**
- **LLMs** are a common feature across various components of AI applications including data pipelines (ETL).

Example health research application:

- Application which uses an LLM to processes academic articles in service of regularly updating a RAG corpus.
- Derives structured data representations from the documents and metadata tags.
- Need to evaluate the LLM output in the pipeline, as well as the main RAG application (eval both retrieval and generation metrics)

Data processing new RAG corpus documents



Properties of Modern AI (GenAI) systems

Modern AI systems typically contain GenAI models (e.g. GPT-4o) and thus these systems have a number of interesting and more complex properties when it comes to eval and testing:

- GenAI applications are **multi-component systems** where changes in one part can affect others in unexpected ways. For instance, a change in your vector database could affect your LLM's responses, or updates to a feature pipeline could impact your machine learning model's predictions.

Properties of Modern AI (GenAI) systems

Modern AI systems typically contain GenAI models (e.g. GPT-4o) and thus these systems have a number of interesting and more complex properties when it comes to eval and testing:

- GenAI applications are **multi-component systems** where changes in one part can affect others in unexpected ways. For instance, a change in your vector database could affect your LLM's responses, or updates to a feature pipeline could impact your machine learning model's predictions.
- GenAI applications are **non-stationary**, meaning their behavior changes over time even if you don't change the code. This happens because the world they interact with changes - new data comes in, language patterns evolve, and third-party models can get silently updated. A test that passes today might fail tomorrow, not because of a bug, but because the underlying conditions have shifted.

Properties of Modern AI (GenAI) systems

Modern AI systems typically contain GenAI models (e.g. GPT-4o) and thus these systems have a number of interesting and more complex properties when it comes to eval and testing:

- GenAI applications are **multi-component systems** where changes in one part can affect others in unexpected ways. For instance, a change in your vector database could affect your LLM's responses, or updates to a feature pipeline could impact your machine learning model's predictions.
- GenAI applications are **non-stationary**, meaning their behavior changes over time even if you don't change the code. This happens because the world they interact with changes - new data comes in, language patterns evolve, and third-party models can get silently updated. A test that passes today might fail tomorrow, not because of a bug, but because the underlying conditions have shifted.
- GenAI applications are **non-deterministic**. Even with the exact same input, they might produce different outputs each time. Think of asking an LLM the same question twice - you might get two different, but equally valid, responses. This makes it impossible to write traditional tests that expect exact matches.

[DISTRIBUTIONAL]

05 GenAI Eval

GenAI Eval

GenAI (and LLMs in particular) has a rapidly developing ecosystem of Eval metrics and frameworks. The Eval model evaluation metrics are less universally understood and more subjective.

While evaluation metrics for LLMs are different than traditional ML, there are some overlapping and analogous goals:

- **Accuracy:** Does it give the correct answer?
- **Factuality:** Are its claims true?
- **Helpfulness:** Is it useful to the user?
- **Safety:** Does it avoid harmful or biased outputs?
- **Generalization:** Can it handle tasks it wasn't explicitly trained on?

Differences:

- The evaluation metrics themselves look very different than in ML. They are functions that take **text as input**. e.g. BLEU, ROUGE

GenAI Eval: Humans in the Loop

Humans (and their opinions) are often involved in the evaluation process, which is very different from how automated evaluations of ML systems works.

Humans create **Golden (validation) datasets**:

- They writing input for these datasets to mimic real life input
- They also can write correct/good responses to example or edit LLM-generated responses
- They can also provide binary/categorical (👍vs 👎) feedback on responses

Limitations:

- Golden datasets evaluated in development might not reflect the true distributions of the input data (how users interact with your system).
- Golden datasets may only reflect certain conditions that you anticipated where as real life usage may have many wild deviations from that.

GenAI Eval: LLM-as-Judge

What is LLM-as-Judge? ⚖️

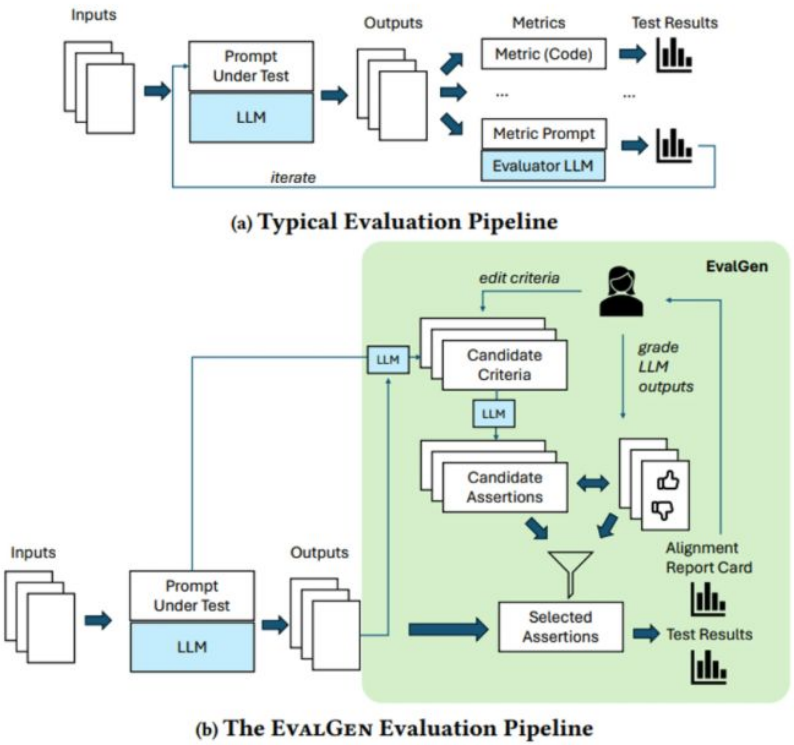
- Definition: Using a LLM to evaluate the quality, accuracy, or usefulness of responses generated by GebAI models.
- Often used to replace or supplement human annotators in large-scale evaluations.
- Fully automated.

Limitations:

- **Bias:** LLMs may favor stylistic features or similar outputs.
- **Prompt sensitivity:** Evaluation quality can vary with prompt design.
- **Validation required:** LLM judgments often need correlation with human evaluations to be trusted.

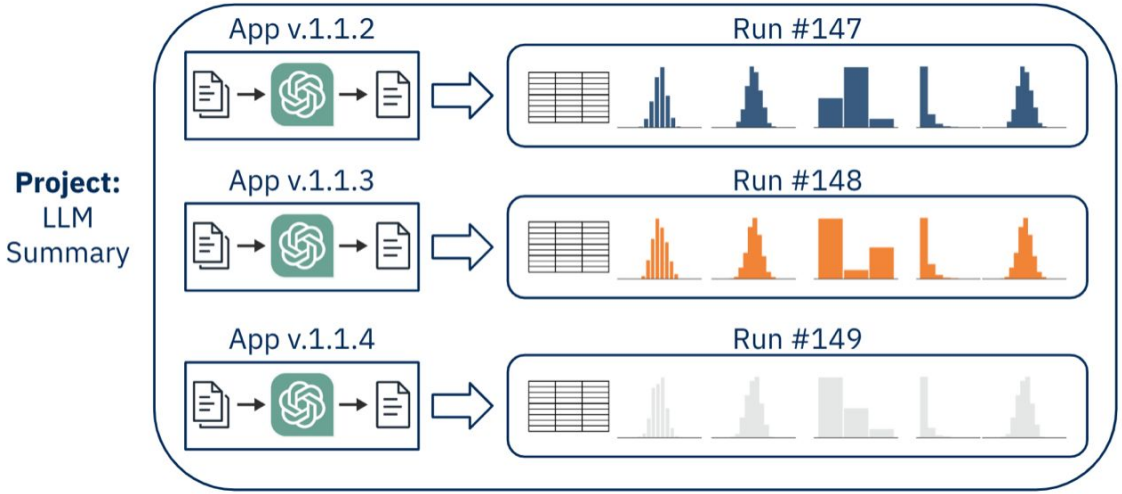
GenAI Eval: EvalGen

- EvalGen is a framework for automating the evaluation of language model outputs using LLMs-as-judges.
- Helps align your evaluation data to human feedback (alignment).
- While generating candidate implementations (Python functions, LLM grader prompts), EvalGen asks humans to grade a subset of LLM outputs; this feedback is used to select implementations that better align with user grades.



GenAI Eval: Distributional Testing in Production

Similar to how you need to test for data drift in the feature inputs and predictions of traditional ML systems, you also need to test input/output distributions over time with GenAI systems in production.



- The output of LLMs is text based (structured outputs can contain other metadata)
- There are many derived metrics from LLM input/output which you can track over time.
- In contrast to ML, where tracking the distribution of a single predicted value is very straightforward.

GenAI Eval: Software

There's a number of **off-the-shelf Eval packages** available in Python and at least one in R!

- **Inspect:** Python library for LLM evals by J.J. Allaire
- **Vitals:** R port of the Inspect library in the Tidyverse by Simon Couch
- **evals:** By OpenAI
- **DeepEval:** Unit testing framework for LLM outputs
- **RAGAS:** Specific to RAG metrics
- **TruLens:** Also focuses on RAG metrics, has a UI

Logging:

- Eval frameworks need logs of the LLM calls (called "traces") to compute metrics.
- So you need some way of logging your inputs/outputs.

[DISTRIBUTIONAL]

06 LLM Eval in R

vitals

lifecycle experimental | CRAN not published | R-CMD-check.yaml passing

vitals is a framework for large language model evaluation in R. It's specifically aimed at ellmer users who want to measure the effectiveness of their LLM products like custom chat apps and querychat apps. You can use it to:

- Measure whether changes in your prompts or additions of new tools improve performance in your LLM product
- Compare how different models affect performance, cost, and/or latency of your LLM product
- Surface problematic behaviors in your LLM product

The package is an R port of the widely adopted Python framework Inspect. While the package doesn't integrate with Inspect directly, it allows users to interface with the Inspect log viewer and provides an on-ramp to transition to Inspect if need be by writing evaluation logs to the same file format.

LLM Eval in R: Vitals

```r
simple_addition <- tibble(
  input = c("What's 2+2?", "What's 2+3?", "What's 2+4?"),
  target = c("4", "5", "6")
)

tsk <- Task$new(
  dataset = simple_addition,
  solver = generate(chat_anthropic(model = "claude-sonnet-4-20250514")),
  scorer = model_graded_qa()
)
```

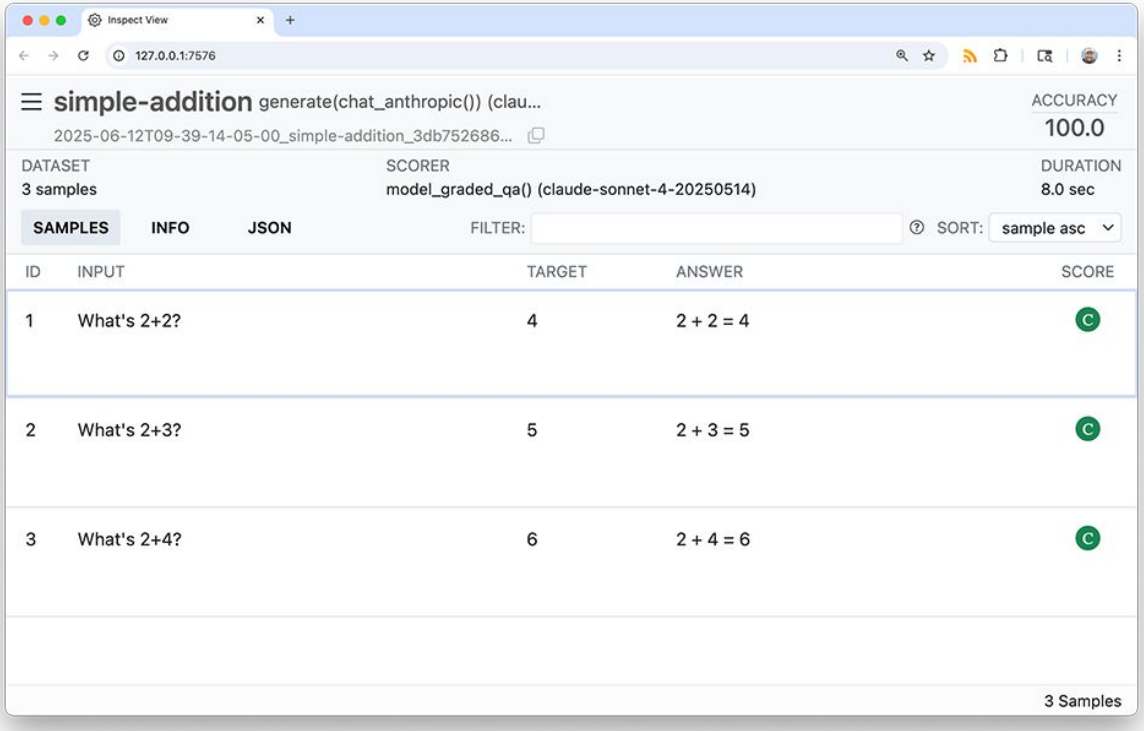
Tasks are composed of three main components:

- **Datasets** are a data frame with, minimally, columns input and target. input represents some question or problem, and target gives the target response.
- **Solvers** are functions that take input and return some value approximating target, likely wrapping ellmer chats. generate() is the simplest scorer in vitals, and just passes the input to the chat's $chat() method, returning its result as-is.
- **Scorers** juxtapose the solvers' output with target, evaluating how well the solver solved the input.

Once you create the evaluation task, then you run it.

LLM Eval in R: Vitals

`$eval()` will run the solver, run the scorer, and then situate the results in a persistent log file that can be explored interactively with the Inspect log viewer.

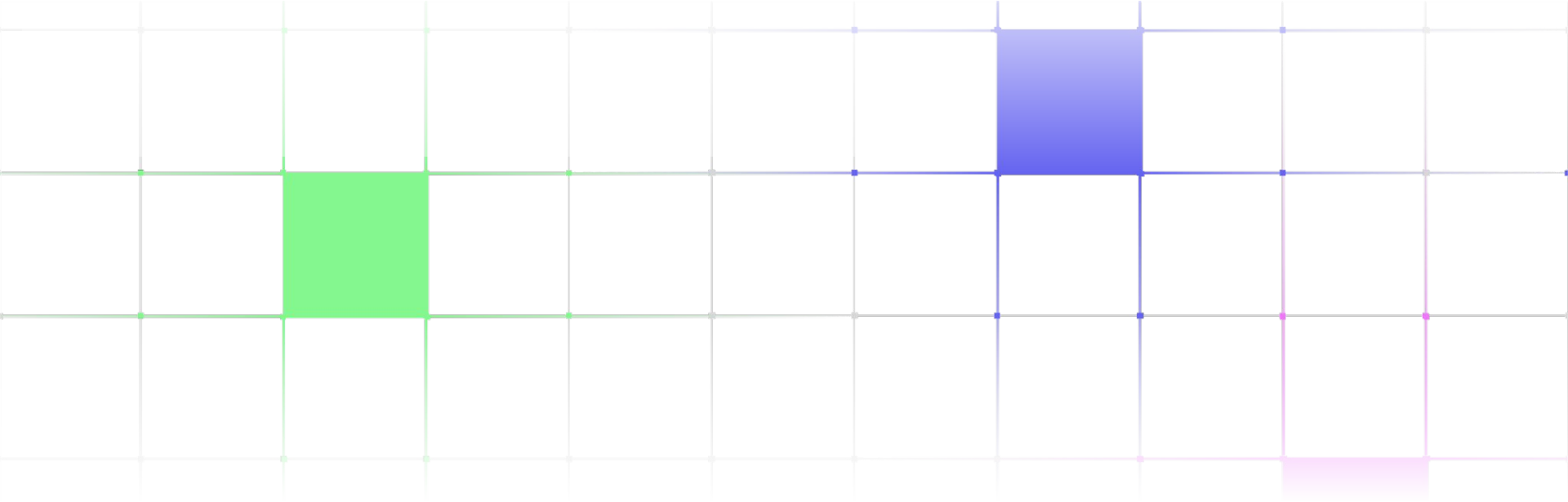


07 Resources

- Vitals R library: https://github.com/tidyverse/vitals
- Hamel Husain: https://hamel.dev/
- Shreya Shankar: https://www.sh-reya.com/
- Hamel & Shreya's LLM Eval course: https://maven.com/parlance-labs/evals

[DISTRIBUTIONAL]

Thank you!

Connect

https://www.linkedin.com/in/erin-ledell/